

1. shell_history.c

```
GNU nano 8.7.1 shell_history.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

#define MAX_HIST 100
#define BUF_SIZE 256

char *history[MAX_HIST];
int history_count = 0;

void enable_raw_mode(struct termios *orig_termios) {
    struct termios raw;
    tcgetattr(STDIN_FILENO, orig_termios);
    raw = *orig_termios;
    raw.c_lflag &= ~(ECHO | ICANON);
    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}
```

```
GNU nano 8.7.1

void disable_raw_mode(struct termios *orig_termios) {
    tcsetattr(STDIN_FILENO, TCSAFLUSH, orig_termios);
}

void add_to_history(const char *cmd) {
    if (strlen(cmd) == 0) return;
    if (history_count < MAX_HIST) {
        history[history_count++] = strdup(cmd);
    }
}

int main() {
    struct termios orig_termios;
    char buffer[BUF_SIZE];
    int pos = 0;
    int history_index = history_count;

    printf("Custom Shell > ");
    fflush(stdout);

    enable_raw_mode(&orig_termios);

    while (1) {
        char ch;
        if (read(STDIN_FILENO, &ch, 1) <= 0) break;
```

```

if (ch == '\033') { // Escape character
    char seq[2];
    if (read(STDIN_FILENO, &seq[0], 1) > 0 && read(STDIN_FILENO, &seq[1], 1) > 0) {
        if (seq[0] == '[') {
            if (seq[1] == 'A') { // UP key
                if (history_index > 0) {
                    history_index--;
                    printf("\r\033[KCustom Shell > %s", history[history_index]);
                    fflush(stdout);
                    strcpy(buffer, history[history_index]);
                    pos = strlen(buffer);
                }
            } else if (seq[1] == 'B') { // DOWN key
                if (history_index < history_count - 1) {
                    history_index++;
                    printf("\r\033[KCustom Shell > %s", history[history_index]);
                    fflush(stdout);
                    strcpy(buffer, history[history_index]);
                    pos = strlen(buffer);
                } else {
                    history_index = history_count;
                    buffer[0] = '\0';
                    pos = 0;
                    printf("\r\033[KCustom Shell > ");
                    fflush(stdout);
                }
            }
        }
    }
}

```

```

    } else if (ch == '\n') { // Enter key
        buffer[pos] = '\0';
        printf("\n");
        disable_raw_mode(&orig_termios);

        if (strcmp(buffer, "exit") == 0) break;

        add_to_history(buffer);
        printf("Executed command: %s\n", buffer);

        pos = 0;
        history_index = history_count;
        printf("Custom Shell > ");
        fflush(stdout);
        enable_raw_mode(&orig_termios);
    } else if (ch == 127 || ch == '\b') { // Backspace
        if (pos > 0) {
            pos--;
            printf("\b \b");
            fflush(stdout);
        }
    } else { // Regular character
        if (pos < BUF_SIZE - 1) {
            buffer[pos++] = ch;
            write(STDOUT_FILENO, &ch, 1);
        }
    }
}

disable_raw_mode(&orig_termios);

```

```

    for (int i = 0; i < history_count; i++) {
        free(history[i]);
    }

    return 0;
}

```

OUTPUT :

```
abhis@MIKHAIL-37:~$ nano shell_history.c
abhis@MIKHAIL-37:~$ gcc -g -Wall shell_history.c -o shell_history
./shell_history
Custom Shell > ls
Executed command: ls
Custom Shell > pwd
Executed command: pwd
Custom Shell > exit
```

2. memory_test.c

```

GNU nano 8.7.1 memory_test.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char **items;
    size_t capacity;
    size_t size;
} DynamicList;

DynamicList* create_list(size_t initial_capacity) {
    DynamicList *list = malloc(sizeof(DynamicList));
    list->capacity = initial_capacity;
    list->size = 0;
    list->items = malloc(list->capacity * sizeof(char*));
    return list;
}

void add_item(DynamicList *list, const char *item) {
    if (list->size >= list->capacity) {
        list->capacity *= 2;
        list->items = realloc(list->items, list->capacity * sizeof(char*));
    }
    list->items[list->size++] = strdup(item);
}

void free_list(DynamicList *list) {
    for (size_t i = 0; i < list->size; i++) {
        free(list->items[i]);
    }
    free(list->items);
    free(list);
}

int main() {
    DynamicList *list = create_list(2);

    add_item(list, "ls -la");
    add_item(list, "cd ..");
    add_item(list, "valgrind --check"); // Triggers dynamic realloc

    for (size_t i = 0; i < list->size; i++) {
        printf("Command [%zu]: %s\n", i, list->items[i]);
    }

    free_list(list);
    return 0;
}

```

OUTPUT :

```
abhis@MIKHAIL-37:~$ nano memory_test.c
abhis@MIKHAIL-37:~$ gcc -g memory_test.c -o memory_test
abhis@MIKHAIL-37:~$ valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./memory_test
==2390== Memcheck, a memory error detector
==2390== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2390== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2390== Command: ./memory_test
==2390==
Command [0]: ls -la
Command [1]: cd ..
Command [2]: valgrind --check
==2390==
==2390== HEAP SUMMARY:
==2390==     in use at exit: 0 bytes in 0 blocks
==2390==   total heap usage: 7 allocs, 7 frees, 1,126 bytes allocated
==2390==
==2390== All heap blocks were freed -- no leaks are possible
==2390==
==2390== For lists of detected and suppressed errors, rerun with: -s
==2390== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```